Informatik I
Dr. Henrik Brosenne
Paul Sarnighausen-Cahn

Wintersemester 2025/26
Georg-August-Universität Göttingen
Institut für Informatik

Übungsblatt 07

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übungen

Abgabe bis Di., 13.01., 14 Uhr

Allgemein

Die Lösung dieser Aufgaben werden über das Lernmodul VIPS abgegeben.

Aufgabe 1 – 14 Punkte

Kategorien von Array-Algorithmen (i1-u-arr1)

In der Vorlesung wurden grundlegende Algorithmen auf Arrays in vier Kategorien eingeteilt: lesende, schreibende, prüfende und umordnende Algorithmen.

Gegeben seien die folgenden Python-Funktionen:

```python
def f1(xs):
    m = xs[0]
    for x in xs:
        if x > m:
            m = x
    return m


def f2(xs):
    for i in range(len(xs)):
        xs[i] = 2 * xs[i]


def f3(xs):
    for i in range(len(xs) - 1):
        if xs[i] > xs[i + 1]:
            return False
    return True
```

Bearbeiten Sie die folgenden Teilaufgaben:

1. Ordnen Sie jede Funktion einer in der Vorlesung verwendeten Kategorien zu.
2. Begründen Sie jede Zuordnung in 2-3 Sätzen.
3. Für welche Funktionen ist ein vorzeitiger Abbruch möglich?
4. Welche Funktionen verändern das Array dauerhaft und warum ist das relevant?

(14 Punkte)

Aufgabe 2 – 12 Punkte

Ein Algorithmus für bergförmige Arrays (i1-u-arr2)

Ein Array heiße bergförmig, wenn seine Werte zunächst streng ansteigen und danach streng abfallen. Beispiel: `[1, 4, 9, 12, 8, 3]`

Ziel ist es, das größte Element (den Gipfel) des Arrays zu bestimmen.

1. Beschreiben Sie in eigenen Worten einen Algorithmus, der das Maximum in einem bergförmigen Array findet, ohne zwangsläufig jedes Element betrachten zu müssen. Sie müssen keinen Code angeben; eine präzise Schritt-für-Schritt-Beschreibung genügt.
2. Erklären Sie, welche Information ein Vergleich zweier benachbarter Elemente liefert und wie diese Information genutzt werden kann.
3. Beschreiben Sie, wie der betrachtete Bereich des Arrays bei jedem Schritt verkleinert wird.
4. Begründen Sie informell, warum Ihr Algorithmus korrekt ist, d. h. warum das Maximum nicht übersprungen werden kann.

Hinweis: Skizzen oder kurze Beispiele dürfen zur Erklärung verwendet werden. Sie können davon ausgehen, dass das Maximum nicht am Rand ist und es genau einen Gipfel gibt.
(12 Punkte)

Aufgabe 3 – 12 Punkte

Zustände bei verschachtelten Schleifen (i1-u-arr3)

Betrachten Sie die folgende Python-Funktion, die auf einem Array ganzer Zahlen arbeitet:

```
def count_smaller(xs):
    count = 0
    for i in range(len(xs)):
        for j in range(i + 1, len(xs)):
            if xs[j] < xs[i]:
                count += 1
    return count
```

Die Funktion verändert das Array nicht, sondern wertet dessen Inhalt aus.

Bearbeiten Sie die folgenden Teilaufgaben:

1. Beschreiben Sie in eigenen Worten, was die Funktion insgesamt berechnet.
2. Erklären Sie präzise, was die Variable `count` nach Abschluss der äußeren Schleife für ein festes `i` repräsentiert, also nach Beendigung der inneren Schleife.
3. Begründen Sie, warum jede relevante Paarung von Array-Elementen genau einmal betrachtet wird.

Hinweis: Es kann hilfreich sein, die Funktion gedanklich an einem kurzen Beispielarray nachzuvollziehen.
(12 Punkte)

Aufgabe 4 – 12 Punkte

Einführung in rekursives Denken (i1-u-rec0)

In Python kann eine Funktion sich selbst aufrufen. Betrachten Sie das folgende Beispiel:

```
def countdown(n):
    if n == 0:
        print("Start!")
    else:
        print(n)
        countdown(n - 1)
```

Bearbeiten Sie die folgenden Teilaufgaben:

1. Beschreiben Sie Schritt für Schritt, was beim Aufruf `countdown(3)` passiert. Geben Sie die Ausgaben in der richtigen Reihenfolge an.
2. Erklären Sie, warum die Bedingung `n == 0` notwendig ist.
3. Was würde passieren, wenn diese Bedingung entfernt würde?

(12 Punkte)

Praktische Übung

Abgabe der Prüfsumme bis Di., 13.01., 14 Uhr
Testat ab Di., 13.01., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 50 Punkte

Fraktale Kurven – Drachenkurve (i1-p-di0)

Durch Zeichenketten, die ausschließlich aus den Buchstaben `F`, `L` und `R` bestehen, lassen sich fraktale Kurven beschreiben.

Dabei haben die einzelnen Zeichen folgende Bedeutung:

- `F`: Bewege den Zeichenstift um die Länge 1 in die Richtung, in die er aktuell orientiert ist, und zeichne dabei eine Linie.
- `L`: Drehe den Zeichenstift um 90° nach links.
- `R`: Drehe den Zeichenstift um 90° nach rechts.

In dieser Aufgabe sollen Sie die sogenannte Drachenkurve erzeugen, simulieren und mithilfe des Moduls `jupyturtle` darstellen.

Drachenkurve

Die Zeichenkette

$$d_0 = \texttt{F}$$

repräsentiert die Drachenkurve der Ordnung 0.

Die Zeichenkette d_{i+1} der Drachenkurve mit Ordnung $i + 1$ entsteht aus der Zeichenkette d_i der Ordnung i wie folgt:

1. Hängen Sie an d_i zunächst das Zeichen `R` an.

2. Hängen Sie anschließend d_i in umgekehrter Reihenfolge an.

3. Ersetzen Sie dabei in der umgekehrten Zeichenkette jedes `L` durch ein `R` und jedes `R` durch ein `L`.

Beispiel:

Ordnung 0:	`F`
Ordnung 1:	`FRF`
Ordnung 2:	`FRFRFLF`
Ordnung 3:	`FRFRFLFRFRFLFLF`

Teil a) Erzeugen der Zeichenkette

Schreiben Sie ein Python-Programm, das die Zeichenkette einer Drachenkurve der Ordnung $1 \leq n \leq 13$ erzeugt.

Gehen Sie dabei iterativ vor.

Schreiben Sie die erzeugte Zeichenkette anschließend in eine Textdatei `dragon.txt`. Die Datei darf ausschließlich die Zeichen `F`, `L` und `R` enthalten. Leerzeichen oder Zeilenumbrüche sind nicht erlaubt.

Testen Sie Ihre Implementierung zunächst mit kleinen Ordnungen (z. B. $0 \leq n \leq 3$) und vergleichen Sie die Ergebnisse mit den angegebenen Beispielen.
(15 Punkte)

Hinweis: Falls Ihre Implementierung nicht korrekt ist, können Sie die bereitgestellten Dateien zum Testen der folgenden Aufgaben nutzen.

Teil b) Simulation der Bewegung

Schreiben Sie eine Funktion, die die Bewegung der Drachenkurve ohne grafische Ausgabe simuliert.

Dabei gelten folgende Vorgaben:

- Die Startposition ist $(0, 0)$.
- Die Anfangsrichtung ist nach rechts.
- Jeder Vorwärtsschritt (`F`) verändert die Position um eine Einheit.

Bestimmen Sie während der Simulation die minimalen und maximalen x- und y-Koordinaten, die erreicht werden.

Diese Simulation dient ausschließlich dazu, Aussagen über die Ausdehnung der Kurve zu treffen.
(15 Punkte)

Teil c) Bestimmung der Fenstergröße

Bestimmen Sie aus den Ergebnissen der Simulation eine geeignete Fenstergröße für die grafische Darstellung.

Gehen Sie dabei wie folgt vor:

- Berechnen Sie aus den minimalen und maximalen Koordinaten eine benötigte Breite und Höhe.

- Verdoppeln Sie sowohl die berechnete Breite als auch die berechnete Höhe.
- Verwenden Sie anschließend als tatsächliche Fenstergröße jeweils das Minimum aus dem berechneten Wert und 600.

Begründen Sie kurz in einem Kommentar, warum eine Begrenzung der Fenstergröße sinnvoll ist, obwohl die Bewegung zuvor simuliert wurde.
(10 Punkte)

Teil d) Zeichnen der Drachenkurve

Lesen Sie die Zeichenkette aus der Datei `dragon.txt` ein und zeichnen Sie die Drachenkurve mithilfe des Moduls `jupyturtle`.
Verwenden Sie dabei:

- eine feste Schrittlänge,
- die zuvor bestimmte Fenstergröße,
- ausschließlich relative Bewegungen des Zeichenstifts.

Versehen Sie Ihren Quelltext mit sinnvollen, erklärenden Kommentaren.

Hinweis: Mit dem folgenden Code können Sie den Zeichenstift versetzen und die Fenstergröße anpassen:

```
# Fenstergröße setzen
t.drawing.width = width
t.drawing.height = height
t.active_pen = False
t.move_to(t.drawing.width // 2, t.drawing.height // 2)
t.active_pen = True
```

(10 Punkte)